COER UNIVERSITY

ROORKEE

Department of Computer Science & Engineering

Labsheet- 04

Unit I — Full Stack Architecture & Planning

Subject / Course	Full Stack Web Development
Unit	I — Full Stack Architecture & Planning
Student Name	Suyash Singh Tomar
Roll No.	60
CU.ID	CU24250178
Semester	V
Faculty Name	Ms. ANKITA
Submission Date	

Academic Year 2026–27

Question 1: Custom Modules

Files: sum.js, even-odd.js, app1.js

Objective: Create two custom modules, export one function from each module, import them into app1.js, and call them with sample values.

sum.js

```
function sum(a, b) {
    return a + b;
}

module.exports = sum;
```

even-odd.js

```
function checkEvenOdd(number) {
    return number % 2 === 0 ? "Even" : "Odd";
}

module.exports = checkEvenOdd;
```

app1.js

```
const sum = require("./sum");
const checkEvenOdd = require("./even-odd");

console.log("Sum of 10 and 20:", sum(10, 20));
console.log("15 is:", checkEvenOdd(15));
console.log("24 is:", checkEvenOdd(24));
```

Run:

```
node app1.js
```

Sample Output:

```
Sum of 10 and 20: 30
15 is: Odd
24 is: Even
```

Explanation:

module.exports makes a function available to another file. require() imports the exported function into app1.js.

Question 2: File System — CRUD on a Text File

Files: file-ops.js

Objective: Create student.txt, write student details, append a course, read the complete file, and finally rename it to profile.txt.

file-ops.js

```
const fs = require("fs");
```

```
const fileName = "student.txt";
const newFileName = "profile.txt";

// Create/write the file
fs.writeFile(fileName, "Name: Suyash\nRoll No: 58\n", (err) => {
    if (err) throw err;
    console.log("student.txt created and written.");

    // Append course
    fs.appendFile(fileName, "Course: Full Stack Web Development\n", (err) => {
        if (err) throw err;
        console.log("Course appended.");

        // Read complete file
        fs.readFile(fileName, "utf8", (err, data) => {
            if (err) throw err;
            console.log("\nFile Content:");
            console.log(data);

            // Rename file
            fs.rename(fileName, newFileName, (err) => {
                if (err) throw err;
                console.log("File renamed to profile.txt");
            });
        });
    });
});
```

Run:

```
node file-ops.js
```

Sample Output:

```
student.txt created and written.
Course appended.

File Content:
Name: Suyash
Roll No: 58
Course: Full Stack Web Development

File renamed to profile.txt
```

Explanation:

fs.writeFile() creates or overwrites a file, fs.appendFile() adds data, fs.readFile() reads it, and fs.rename() changes the filename.

Question 3: Directory Operations

Files: dir-ops.js

Objective: Create an uploads directory, create three empty files, list the files, and delete one file.

dir-ops.js

```
const fs = require("fs");

const dirName = "uploads";

if (!fs.existsSync(dirName)) {
    fs.mkdirSync(dirName);
    console.log("uploads directory created.");
} else {
    console.log("uploads directory already exists.");
}

const files = ["file1.txt", "file2.txt", "file3.txt"];

files.forEach((file) => {
    fs.writeFileSync(`${dirName}/${file}`, "");
});

console.log("Three files created.");

fs.readdir(dirName, (err, fileList) => {
    if (err) throw err;

    console.log("Files in uploads:");
    console.log(fileList);

    fs.unlink(`${dirName}/file2.txt`, (err) => {
        if (err) throw err;
        console.log("file2.txt deleted.");
    });
});
```

Run:

```
node dir-ops.js
```

Sample Output:

```
uploads directory created.
Three files created.
Files in uploads:
[ 'file1.txt', 'file2.txt', 'file3.txt' ]
file2.txt deleted.
```

Explanation:

fs.mkdir() creates a directory, fs.writeFile() creates files, fs.readdir() lists directory contents, and fs.unlink() deletes a file. The exists check prevents an error if uploads already exists.

Question 4: Path Module

Files: path-demo.js

Objective: Use Node.js path functions to display a directory name, base filename, extension, and an absolute path.

path-demo.js

```
const path = require("path");

const filePath = "/home/user/data/report.pdf";
const relativePath = "./data/report.pdf";

console.log("Directory:", path.dirname(filePath));
console.log("Base name:", path.basename(filePath));
console.log("Extension:", path.extname(filePath));
console.log("Absolute path:", path.resolve(relativePath));
```

Run:

```
node path-demo.js
```

Sample Output:

```
Directory: /home/user/data
Base name: report.pdf
Extension: .pdf
Absolute path: /current/working/directory/data/report.pdf
```

Explanation:

The exact path.resolve() output depends on the directory from which the command is executed. path.dirname() returns the directory, path.basename() returns the final filename, and path.extname() returns the extension.

Question 5: OS Module Report

Files: sys-report.js

Objective: Display operating-system information including platform, architecture, memory, CPU cores, and uptime.

sys-report.js

```
const os = require("os");

console.log("Platform:", os.platform());
console.log("Architecture:", os.arch());
console.log("Total Memory (GB):", (os.totalmem() / (1024 ** 3)).toFixed(2));
console.log("Free Memory (GB):", (os.freemem() / (1024 ** 3)).toFixed(2));
console.log("CPU Cores:", os.cpus().length);
console.log("System Uptime (minutes):", (os.uptime() / 60).toFixed(2));
```

Run:

```
node sys-report.js
```

Sample Output:

```
Platform: win32
Architecture: x64
Total Memory (GB): 15.90
Free Memory (GB): 7.42
```

```
CPU Cores: 8
System Uptime (minutes): 125.50
```

Explanation:

The sample memory, CPU count, platform, and uptime values are machine-dependent. os.platform(), os.arch(), os.totalmem(), os.freemem(), os.cpus(), and os.uptime() provide these values.

Question 6: Events Module — Custom EventEmitter

Files: order-events.js

Objective: Create a custom EventEmitter, listen for orderPlaced, and print a receipt whenever an order is placed.

order-events.js

```
const EventEmitter = require("events").EventEmitter;

const orderEmitter = new EventEmitter();

orderEmitter.on("orderPlaced", (order) => {
    console.log("\n----- ORDER RECEIPT -----");
    console.log("Order ID:", order.id);
    console.log("Item:", order.item);
    console.log("Amount: Rs.", order.amount);
    console.log("-------------------------");
});

const orders = [
    { id: 101, item: "Keyboard", amount: 1200 },
    { id: 102, item: "Mouse", amount: 700 },
    { id: 103, item: "Headphones", amount: 1800 }
];

orders.forEach((order) => {
    orderEmitter.emit("orderPlaced", order);
});
```

Run:

```
node order-events.js
```

Sample Output:

```
----- ORDER RECEIPT -----
Order ID: 101
Item: Keyboard
Amount: Rs. 1200
-------------------------

----- ORDER RECEIPT -----
Order ID: 102
Item: Mouse
Amount: Rs. 700
-------------------------
```

```
----- ORDER RECEIPT -----
Order ID: 103
Item: Headphones
Amount: Rs. 1800
-------------------------
```

Explanation:

EventEmitter allows an application to define custom events. on() registers a listener, while emit() triggers the event and passes the order object to the listener.

Question 7: HTTP Server with Manual Routing

Files: server.js

Objective: Create a framework-free HTTP server with routes for the home page, all students, an individual student, and a 404 response.

server.js

```
const http = require("http");
const url = require("url");

const students = [
    { id: 1, name: "Rahul", course: "CSE" },
    { id: 2, name: "Suyash", course: "CSE" },
    { id: 3, name: "Aman", course: "CSE" }
];

const server = http.createServer((req, res) => {
    const parsedUrl = url.parse(req.url, true);
    const pathname = parsedUrl.pathname;

    if (req.method === "GET" && pathname === "/") {
        res.writeHead(200, { "Content-Type": "text/plain" });
        res.end("Welcome to the Node.js Student Server!");
        return;
    }

    if (req.method === "GET" && pathname === "/students") {
        res.writeHead(200, { "Content-Type": "application/json" });
        res.end(JSON.stringify(students));
        return;
    }

    if (req.method === "GET" && pathname.startsWith("/students/")) {
        const id = Number(pathname.split("/")[2]);
        const student = students.find((s) => s.id === id);

        if (student) {
            res.writeHead(200, { "Content-Type": "application/json" });
            res.end(JSON.stringify(student));
        } else {
            res.writeHead(404, { "Content-Type": "application/json" });
            res.end(JSON.stringify({ error: "Student not found" }));
        }
        return;
```

```
    }

    res.writeHead(404, { "Content-Type": "text/html" });
    res.end("<h1>404 - Page Not Found</h1>");
});

server.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
});
```

Run:

```
node server.js
```

Sample Output:

```
Server running at http://localhost:3000

Test routes:
GET http://localhost:3000/
GET http://localhost:3000/students
GET http://localhost:3000/students/2
GET http://localhost:3000/unknown
```

Explanation:

The server uses the built-in http module. Manual routing checks req.method and the pathname. Student IDs are extracted by splitting /students/:id. Stop the server with Ctrl+C.

Question 8: Query String Parsing

Files: query-demo.js

Objective: Read the keyword query parameter from a URL such as /search?keyword=node and return it in the response.

query-demo.js

```
const http = require("http");
const url = require("url");

const server = http.createServer((req, res) => {
    const parsedUrl = url.parse(req.url, true);

    if (parsedUrl.pathname === "/search") {
        const keyword = parsedUrl.query.keyword || "";

        res.writeHead(200, { "Content-Type": "text/plain" });
        res.end(`Search keyword: ${keyword}`);
        return;
    }

    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("404 - Route not found");
});

server.listen(3001, () => {
```

```
    console.log("Search server running at http://localhost:3001");
});
```

Run:

```
node query-demo.js
```

Sample Output:

```
Search server running at http://localhost:3001

Open:
http://localhost:3001/search?keyword=node

Response:
Search keyword: node
```

Explanation:

url.parse(req.url, true) parses the URL and its query string. The query object is available through parsedUrl.query, so keyword is obtained using parsedUrl.query.keyword.

Question 9: Reading a Large File with Streams

Files: stream-demo.js

Objective: Create a text file with 50 lines and read it using a readable stream while displaying the size of every received chunk.

stream-demo.js

```
const fs = require("fs");

let content = "";

for (let i = 1; i <= 50; i++) {
    content += `This is line ${i} of the sample large file.\n`;
}

fs.writeFileSync("large.txt", content);
console.log("large.txt created with 50 lines.");

const stream = fs.createReadStream("large.txt", { encoding: "utf8" });

stream.on("data", (chunk) => {
    console.log("Chunk received:", Buffer.byteLength(chunk, "utf8"), "bytes");
});

stream.on("end", () => {
    console.log("Finished reading large.txt");
});

stream.on("error", (err) => {
    console.error("Error:", err.message);
});
```

Run:

```
node stream-demo.js
```

Sample Output:

```
large.txt created with 50 lines.
Chunk received: 1024 bytes
Chunk received: 1024 bytes
Chunk received: 1024 bytes
Chunk received: 421 bytes
Finished reading large.txt
```

Explanation:

The exact number and size of chunks can vary by Node.js version, operating system, file size, and stream buffering. The data event fires for each chunk, while end fires after the complete file has been read.

Question 10: npm and an External Package

Files: id-gen.js

Objective: Initialize an npm project, install nanoid version 3, and generate five unique IDs.

Commands

```
npm init -y
npm install nanoid@3
```

id-gen.js

```
const { nanoid } = require("nanoid");

for (let i = 1; i <= 5; i++) {
    console.log(`ID ${i}: ${nanoid()}`);
}
```

Run:

```
node id-gen.js
```

Sample Output:

```
ID 1: V1StGXR8_Z5jdHi6B-myT
ID 2: Q8fJ2aP3xK7LmN9rT0uVw
ID 3: zY4kP8sL2mN6qR1tX5cVa
ID 4: A7bC9dE2fG4hJ6kL8mPqR
ID 5: nM3xZ6vB8cD1eF5gH7jKp
```

Explanation:

The IDs above are examples. Your five IDs will be different because nanoid generates random unique identifiers. nanoid@3 is used as requested.

Question 11: Command-Line Arguments

Files: calc-cli.js

Objective: Accept two numbers and an operator through process.argv and perform addition, subtraction, multiplication, or division.

calc-cli.js

```
const args = process.argv.slice(2);

if (args.length !== 3) {
    console.log("Usage: node calc-cli.js <number1> <number2> <operator>");
    process.exit(1);
}

const num1 = Number(args[0]);
const num2 = Number(args[1]);
const operator = args[2];

if (Number.isNaN(num1) || Number.isNaN(num2)) {
    console.log("Please enter valid numbers.");
    process.exit(1);
}

let result;

switch (operator) {
    case "+":
        result = num1 + num2;
        break;
    case "-":
        result = num1 - num2;
        break;
    case "*":
        result = num1 * num2;
        break;
    case "/":
        if (num2 === 0) {
            console.log("Error: Division by zero is not allowed.");
            process.exit(1);
        }
        result = num1 / num2;
        break;
    default:
        console.log("Unsupported operator. Use +, -, *, or /.");
        process.exit(1);
}

console.log(`${num1} ${operator} ${num2} = ${result}`);
```

Run:

```
node calc-cli.js 10 5 +
node calc-cli.js 10 5 -
node calc-cli.js 10 5 "*"
node calc-cli.js 10 5 /
```

Sample Output:

```
10 + 5 = 15
10 - 5 = 5
```

```
10 * 5 = 50
10 / 5 = 2
```

Explanation:

process.argv contains the command-line arguments. slice(2) removes the Node.js executable path and the script filename, leaving the user-provided arguments.

Question 12: Environment Variables

Files: env-demo.js

Objective: Read PORT from process.env, use 4000 when PORT is not defined, and start an HTTP server on the selected port.

env-demo.js

```
const http = require("http");

const PORT = Number(process.env.PORT) || 4000;

const server = http.createServer((req, res) => {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end(`Server is running on port ${PORT}`);
});

server.listen(PORT, () => {
    console.log(`HTTP server started on port ${PORT}`);
});
```

Run:

```
Linux/Mac:
PORT=5000 node env-demo.js

Windows CMD:
set PORT=5000 && node env-demo.js

Windows PowerShell:
$env:PORT=5000; node env-demo.js
```

Sample Output:

```
HTTP server started on port 5000

Open:
http://localhost:5000
```

Explanation:

process.env contains environment variables supplied by the operating system or shell. If PORT is missing or invalid as a number, the expression falls back to port 4000.